DeepL
Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

IoT Security Architecture and Security of LoRa Wireless Networks, Authentication, Confidentiality, Integrity, LoRaWAN, OTAA, ABP.	INSA-Lyon Telecommunications Department Walid Bechkit

Objectives: This practical session has four main objectives:

- To set up and understand simple LoRa communication between a transmitter and a receiver (getting to grips with the LoRa radio protocol).
- Understanding and setting up the three basic security services: **authentication**, **integrity,** and **confidentiality**.
- Understanding and learning how to use the LoRaWAN architecture and the associated LoRaWAN protocol.
- Understanding and implementing LoRaWAN OTAA and ABP security modes.

Keywords: *MKR WAN 1310, LoRa, power control, spreading factor, coding rate, CRC, basic security services, authentication, integrity, confidentiality, LoRaWAN, Device, Gateway, Network Server, Application Server, LoRaWAN frame format, TTN, OTAA, ABP, Hash, MIC, etc.*

Preamble: MKR WAN 1310 board

During this practical, we will use Arduino MKR WAN 1310 boards. The Arduino MKR WAN 1310 board offers low-power LoRa/LoRaWAN connectivity. It includes a low-power Microchip SAMD21 processor, a Murata CMWX1ZZABZ LoRa module, a crypto chip (ECC508), and 2 MB of flash memory.

Start by downloading the latest version of Arduino for Linux 64-bit from the Arduino website. Connect your MKR 1310 boards, **launch Arduino as root**, and if you haven't already done so, install the Arduino SAMD Boards (Cortex-M0+) package, which supports the MKR family of boards, among others. Finally, install the latest version of the LoRa library "by Sandeep Mistry" (for parts 1 and 2) and MKRWAN (for parts 3 and 4).

1 LoRa: getting started and understanding radio parameters



FIGURE 1 – Architecture of parts 1 and 2: simple LoRa communication between a transmitter and a receiver

During the first two parts of this practical assignment, we will focus mainly on the LoRa radio protocol and implement basic security services between a transmitter and a receiver. To do this, we will set up a simple architecture with a transmitter and a receiver communicating using the LoRa protocol. In order to implement LoRa communication, we will use the LoRa library that you have just installed. This library is widely used and is compatible with the MKRWAN 1300 and 1310 as well as cards based on Semtech SX1276/77/78/79 LoRa modules (Dragino Lora Shield, HopeRF RFM95W, RFM96W, RFM98W, etc.). Other more comprehensive libraries exist, such as RH_RF95.h, which is compatible with Grove LoRa modules.

Download the two TP files: TX.ino and RX.ino. Carefully read and understand these programs, which handle LoRa transmission and reception.

1.1 Frequency selection and initial test

The *void LoRa.begin(frequency)* function initializes the LoRa module on the carrier frequency. In the TX and RX programs, update the center frequency (carrier). To reduce interference between different groups, randomly select one of the following frequencies: 867.3 MHz,
867.5 MHz, 867.7 MHz, 867.9 MHz, and 868.1 MHz. Then load the TX program into one Arduino node and the Rx program into another. What do we observe? Are the messages being received correctly?

The "setTxPower" function allows you to set the transmission power level. The txPower values can range from 2 to 20 dBm.

```
LoRa.setTxPower(txPower);
```

For the rest of this practical exercise, let's set the transmission power to 10dBm = 10 mW.

Students who are not taking an in-depth course on LoRa/LoRaWAN and who wish to deepen their understanding of the LoRa physical layer parameters can perform the effective throughput performance evaluation proposed in the following two subsections. For others, you can skip directly to the section "LoRa physical message structure."

1.2 [Bonus] Transmission time measurement

The *unsigned long micros()* function returns the current time in microseconds. In the TX program, save the current time in the variable "Temps_debut_tx" just before calling the Lora.endPacket function, which starts packet transmission. Immediately after this function, calculate the transmission time in milliseconds in the variable Temps_total_tx. Then display this time with the success or failure message.

Examples:

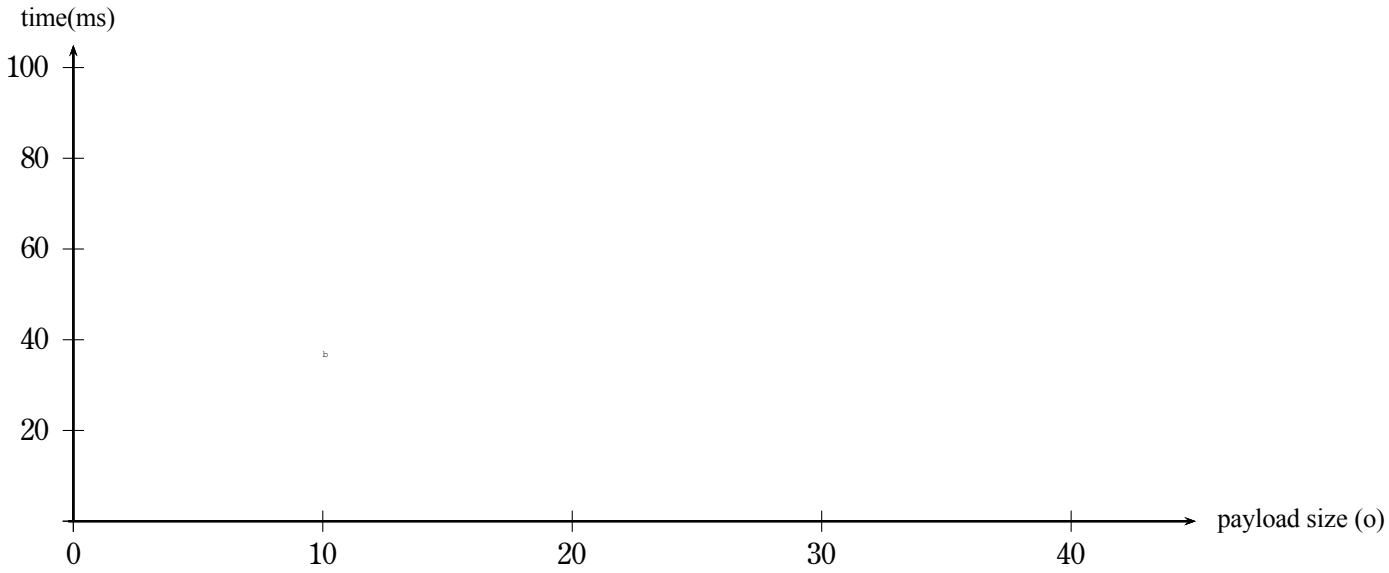
Package transmission: 10... Success ... Transmission time: 36.54 ms
Package transmission: 11... Failure ... after: 100.2 ms

Once the TX.ino program has been updated, launch the Serial Monitor and view the results.

Do you think this time will be sufficient to calculate the transmission rate in bits/s? If so, calculate this rate; if not, why not?

Now change the size of the packet to be sent in bytes (the variable "nb_octets_a_envoyer") by testing the values 10, 20, 30, and 40 bytes. Each time, check the message received and note the transmission time in the table below.

Packet size	10	20	30	40
Time				



Now estimate the effective throughput for this default configuration.

1.3 [Bonus] LoRa physical layer parameters

1.3.1 Spectrum spreading factor

```
LoRa.setSpreadingFactor(spreadingFactor);
```

The above function allows you to set the SF value between 6 and 12 (default value is 7).

1.3.2 Channel width (bandwidth)

The channel width around the center frequency is another very important parameter that affects the transmission rate.

```
LoRa.setSignalBandwidth(signalBandwidth);
```

The above function allows you to set the channel width value, which can take the values 7.8E3, 10.4E3, 15.6E3, 20.8E3, 31.25E3, 41.7E3, 62.5E3, 125E3 (default value), and 250E3. To study the impact of this parameter, we will try (by setting the SF to 8, for example) the widths 62.5E3 (62.5 kHz), 125E3 (125 kHz, which is the default value), and 250E3 (kHz).

1.3.3 Coding Rate C/R

The coding rate is a very important parameter for detecting and correcting transmission errors. The possible C/R values with LoRa are 4/5, 4/6, 4/7, and 4/8. A C/R of 4/8 means that we send twice as many bits as the original message.

```
LoRa.setCodingRate4(codingRateDenominator);
```

The above instruction sets the C/R denominator, which can take values of 5 (default), 6, 7, or 8. (The numerator is set to 4.)

1.4 LoRa physical message structure

LoRa offers two packet format modes:

o The explicit header mode includes a short header containing information on the payload length, the C/R, and whether or not there is a CRC. This mode is the default mode and is the one we have been using up to now.

Preamble (8s)	Physical header (0-3o) Length; C/R; CRC?	Payload (max = 255o)	CRC (0-2o)

o The implicit header mode, in which the payload size, coding rate, and presence of CRC are fixed. In this case, this information is not sent, which reduces the size of the data sent. However, this requires these parameters to be configured manually on both sides.

Preamble (8s)	Payload (max = 255o)	CRC (0-2o)

The library used during this practical exercise uses explicit mode by default. However, any packet can be sent in implicit mode (without physical layer headers) using the command:

```
LoRa.beginPacket(true);
```

In the following, we will continue to use explicit mode without modifying our programs. However, if time allows, you can test explicit mode by updating the programs on both sides.

The use of CRC is disabled by default in the library we are using during this practical. The following two instructions enable (or disable) the use of CRC.

```
LoRa.enableCrc();
LoRa.disableCrc();
```

1.5 Payload formatting

As you have noticed, LoRa physical messages do not allow you to identify the sources and recipients of physical messages sent, nor to manage multiple access to a transmission channel. In order to introduce LoRaWAN, let's start by defining a Level 2 frame structure that contains the source node address, the destination node address, a frame number, and the useful temperature and humidity data. To simplify the task, we define node addresses on a single byte.

In both the TX and RX files, declare the following structure at the very beginning of your file (ideally after the includes or in another header file that you will need to include):

```
typedef struct
{
  byte        destination;
  byte        source;
  int         frame_number;
  float       temperature;
  float       humidity;
} Frame_t;
```

At the transmitter level, declare a variable of type Frame_t (e.g., Frame_t my_frame). Just before sending the packet, fill in the various fields of this structure (e.g., my_frame.temperature= 25.76), and replace the previous LoRa.write() instruction with:

```
LoRa.write( (byte* )& my_frame, sizeof(my_frame) );
```

where ma_trame is your structure containing the packet to be sent.

At the receiver level, declare a variable of type Frame_t (e.g., Frame_t my_frame). Then replace the loop that reads the received bytes one by one with:

```
LoRa.readBytes( (byte*)& my_frame, sizeof(my_frame));
```

Display the fields of the received structure. Ensure that all information has been received correctly.

2 Confidentiality, authentication, and integrity

During this part, only LoRa communications will be set up (LoRa library) and we will implement authentication, integrity, and confidentiality services ourselves using AES-128 symmetric encryption. The planned implementations are deliberately similar to the LoRaWAN standard in order to fully understand the design of LoRaWAN solutions.

2.1 Authentication

The objective of this section is to complete a simple exchange between a client and a server to understand authentication.

- Start by downloading the programs provided.
- The trainer will explain the authentication protocol that has been designed. This protocol is very simple and roughly follows the logic of the exchanges provided for in the LoRaWAN protocols.
- Read the programs and complete the few missing lines in the server program. Test the client and server programs and validate the authentication behavior.

Change a single bit in the key used at the server level. Restart the client to confirm that authentication fails in this case.

In this case, observe the impact on the MIC sent.

Do these exchanges guarantee server authentication to the client? What is the risk in this case? Without implementing it, propose an extension to solve this problem.

2.2 Key generation

Uncomment the part where the encryption session key is generated and installed on both sides: client and server. Based on this part, add another one that allows you to generate and install the integrity key following the protocol explained at the beginning.

2.3 Confidentiality and Integrity

In order to understand the independence of the two features, we will set up data transmissions: i) unencrypted and without integrity, ii) encrypted and without integrity, iii) unencrypted with integrity code, and finally encrypted with integrity code.

Uncomment the part where data is sent. Which scenario does this transmission correspond to?

Using this section as a guide, complete the programs to enable data to be sent using the three other scenarios mentioned above.

On the server side, complete the program so that the MIC can be verified upon receipt in both scenarios with integrity. Test these programs.

At the server level, complete the program to enable decryption of encrypted messages received in both scenarios where data is encrypted. Test these programs.

3 Introduction to LoRaWAN: Architectures and implementations

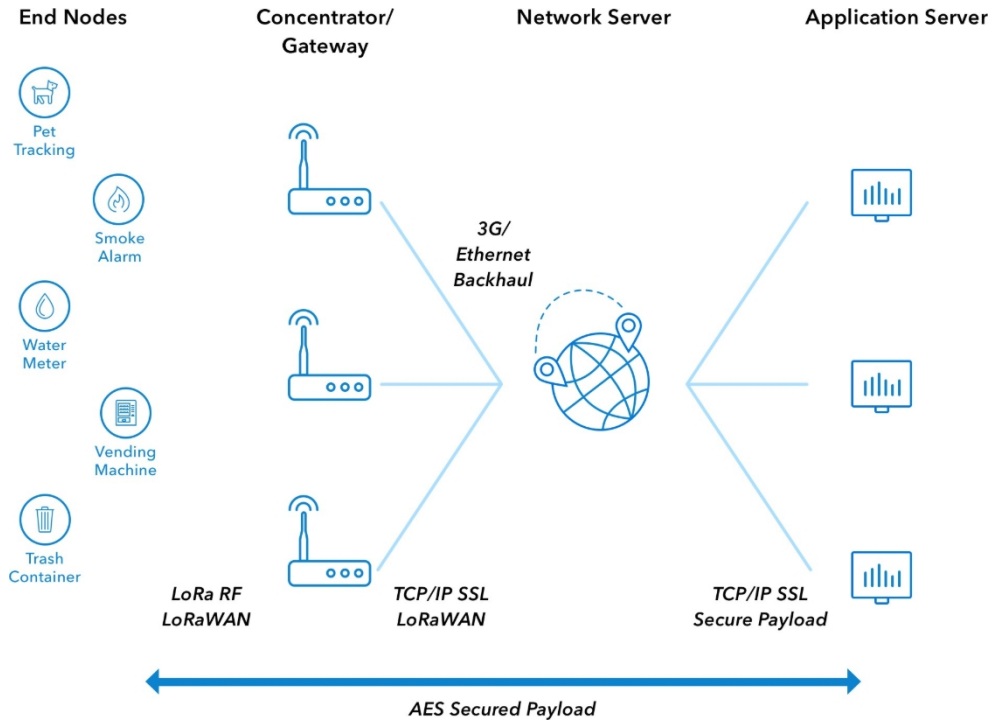


FIGURE 2 – Architecture of a LoRaWAN network (source: www.thethingsnetwork.org)

3.1 Overall architecture

As illustrated in Figure 2, the architecture of LoRaWAN networks is based on four elements:

- **Node (LoRaWaN Device)**: An endpoint device whose main role is to transmit data (often from sensors) upstream to the network. Depending on requirements, these nodes can also receive data downstream, which distinguishes them into three classes (A, B, and C).
- **Gateway (LoRaWAN Gateway)**: Equipment that connects devices to the LoRaWAN core network (network servers and application). It acts as a bridge between devices and The Things Network. It communicates with objects using the low-energy LoRa protocol, while often connecting to the core network using high-speed protocols such as WiFi, Ethernet, or 4G/5G networks. A single gateway can serve thousands of nodes.
- **Network server:** The network server receives all messages from the gateways connected to it. Its role includes i) verifying the integrity of the data received, ii) deduplicating messages (details in TP4), iii) liaising with the application server, iv) selecting the best gateway for transmitting messages downstream.
- **Application server:** Its role is to decrypt messages and present them to users. Post-processing, storage, and analysis of this data can be integrated into the application server but are not part of the LoRaWAN standard.

3.2 Important identifiers and keys to know

The main identifiers you need to know to understand and set up a LoRaWAN network are as follows:

- **DevEUI**: 64-bit physical identifier of the node (device), EUI-64, globally unique, assigned at manufacture
- **DevAddr**: 32-bit node address: dynamic, non-unique, assigned through the activation procedure.
- **GatewayEUI**: 64-bit physical identifier of the gateway, EUI-64 (unique)
- **AppEUI**: 64-bit application identifier, EUI-64, unique per network

In addition, the following keys are also very important to know (details will be provided during the next practical session):

- **AppKey** Primary key used to derive session keys
- **AppSKey**: Session confidentiality key (data encryption)
- **NwkSKey** Data integrity session key

3.3 Different implementations of LoRaWAN networks

3.3.1 Operated networks

Operators are actively launching LoRaWAN network deployments. In France, Orange and Objenious (Bouygues) are deploying and operating large-scale networks with significant coverage. The components—gateways, network servers, and application servers—are managed by the operators, and users only have to manage the end nodes. Generally, a web interface and/or application is provided to configure devices, applications, etc.

3.3.2 Private networks

Any company, community, or individual can easily develop and set up their own LoRaWAN network. Various types of LoRaWAN gateways can be configured (from simple gateways based on Raspberry Pi and the RHF0M301 module to more professional gateways such as LORIX One). Then, all you need to do is install a network server and an application server on simple machines or more powerful servers. Several versions of LoRaWAN servers are available as open source.

3.3.3 Community Networks

Several community networks are being set up around the world, the largest of which is TTN (The Things Network). TTN is a community-based, open-source LoRaWAN network that now has more than 40,000 contributors from 90 countries (4,000 gateways).

3.4 Nodes (LoRaWAN devices): initialization and identification of the DevEUI

As explained above, we will use the MKRWAN library, which implements the LoRaWAN protocol, during this practical exercise. Start by installing this library if it is not already installed (Tools -> Manage Libraries). Download the code provided (Part 3_4) *which simply* allows *you* to initialize the LoRaWAN module and *display* The

begin instruction allows you to initialize the LoRaWAN module by taking the frequency band as a parameter Among the following: AS923, AU915, EU868 (default value), KR920, IN865, US915, and US915_HYBRID.

```
modem.begin(band);
```

Should the value used in the programs provided be changed? If so, modify it and reload the programs. Note the "device EUI" values of your two sensors.

Sensors	Sensor 1 Group ..	Sensor 2 Group ..
Device EUI	A8 61 0A ..	A8 61 0A ..

3.5 Configuring devices and registering them with TTN

Start by creating an account on TTN (each participant or each pair). To do this, log on to https://www.thethingsnetwork.org/https ://www.thethingsnetwork.org/ and fill in the form under "Sign Up."

Now create a new application on TTN and register your sensor node (device) with the TTN application server in the application you have created. To do this, you will need your DevEUI. Note that the Device ID requested will never be exchanged on the LoRaWAN network and is only used by the application server.

Analyze the results displayed on the MKR serial monitor on one side and on the gateway and application on the other. What do you observe?

As seen in class, once the sensor nodes are authenticated and connected to the network, the network assigns them a DevAddr.

```
modem.getDevAddr();
```

Using the above command, display and note the DevAddr of your two sensor nodes. Will they still be the same if we request a connection to this network again after a few hours?

The messages sent by the proposed programs ("welcome to the IoT security class") are simple text messages (ASCII encoding). We can, of course, send other types of data as we did previously.

To verify that the message sent is received by the application, copy the received payload **to the application** and decode it using any base64 decoder. What do we observe? Is the transmitted message received correctly?

Repeat the same steps **on a gateway** if you have access to a TTN gateway. What do we see? Can we decode the payloads on the gateway? Why?

4 LoRaWAN security

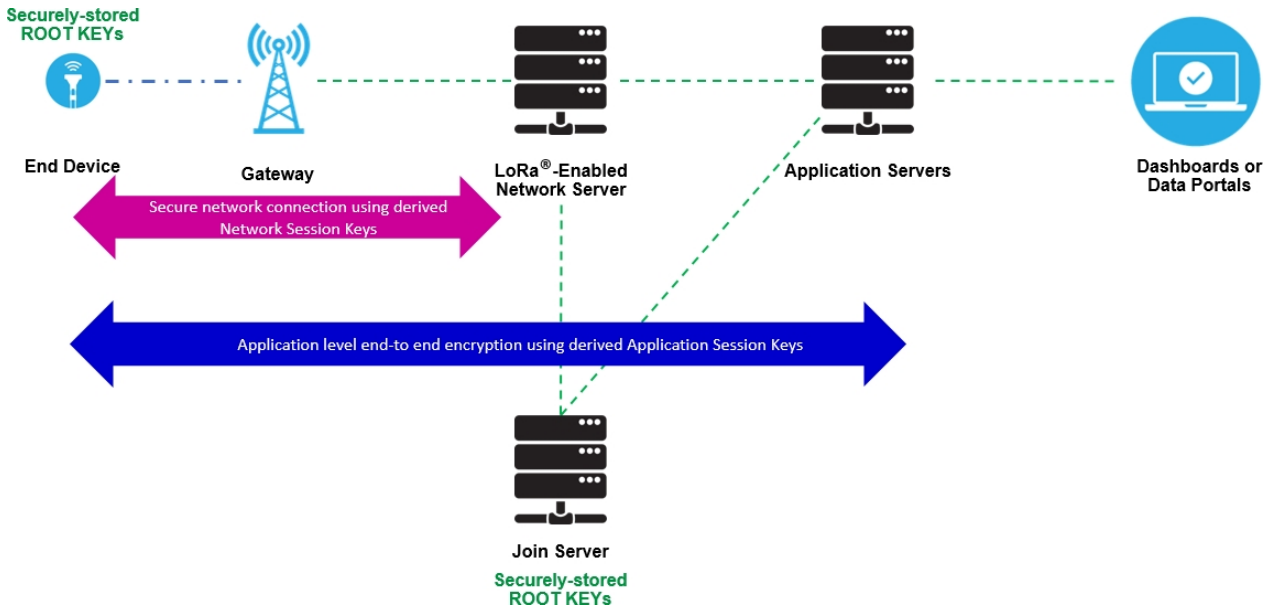


FIGURE 3 – LoRaWAN security architecture (source: https://lora-developers.semtech.com/library/tech-papers-and-guides/lora-and-lorawan/)

LoRaWAN operates in OTTA mode by default. Restart the sensors and check that the data is being received by your application.

4.1 OTTA mode: Over The Air Activation

```
String getAppSKey()
String getNwkSKey();
```

Display the two session keys (AppSKey and NwkSKey) on both sides: at the sensor level (functions above) and at the application server level. Verify that the keys are identical each time.

Based on the first part and the course, explain the role of these two keys and the entities where they are used.

4.2 ABP mode: Activation By Personalization

Start by deleting the nodes registered on TTN and re-registering them in ABP mode. To activate the nodes in ABP mode, the MKRWAN library offers the following instruction:

```
int joinABP(const char * devAddr, const char * nwkSKey, const char * appSKey)
```

Why does this activation require specifying the devAddr in addition to the two session keys?

Now register the nodes using ABP mode. Update the Arduino programs so that this works. Test this mode and comment on it.

Please do not forget to deregister all LoRaWAN Gateways and Devices used during this practical. That's it! You're done. Thank you very much and all the best for the future.

References

— The Things Network: https://www.thethingsnetwork.org/https ://www.thethingsnetwork.org/

— Jansen C. Liando, Amalinda Gamage, Agustinus W. Tengourtius, and Mo Li. Known and Unknown Facts of LoRa: Experiences from a Large-scale Measurement Study. ACM Trans. Sen. Netw. 15, 2, 2019

— Dedicated networks for IoT: PHY/MAC state of the art and challenges C. Goursaud, JM. Gorce, EAI Endorsed Transactions on the Internet of Things, 2016

— LoRa / LoRaWAN training course, Sylvain MONTAGNY, Savoie Mont Blanc University

— Characterization of LoRa modulation and demodulation processes, J-F SERREAU, 2019

— IMA4 P6 Project: LoRaWAN, Gosse Antoine 2018. https://projets-ima.plil.fr/mediawiki/images/f/f6/Rapport_P6_LoR
_Gosse_Antoine.pdf

— https://wiki.seeedstudio.com/LoRa_LoRaWan_Gateway_Kit/

— https://lora.readthedocs.io/en/latest/

— https://get2hub1.blogspot.com/2020/04/available-iot-network-technology.html.

— LoRa library documentation, Sandeep Mistry, available online at:
https://github.com/sandeepmistry/arduino-LoRa/blob/master/API.md

— Characterization of LoRa modulation and demodulation processes, J-F SERREAU, 2019

— http://www.onditech.com/documents/freq_433_868.pdf

— https://www.mobilefish.com/download/lora/lora_part17.pdf